

Лабораторна робота №2

Проектування архітектури програмного забезпечення та моделювання даних

1. Обґрунтування вибору технологічного стеку (Technology Stack)

Мова програмування Back-end:

Обрано Next.js (Node.js runtime), оскільки платформа дозволяє реалізувати серверну логіку в межах одного проєкту разом із клієнтською частиною.

Next.js підтримує створення API-маршрутів (API Routes), що дає можливість реалізувати:

- систему автентифікації та авторизації (JWT)
- рольову модель доступу (Пацієнт, Лікар, Адміністратор)
- логіку бронювання прийомів
- модуль рекомендацій лікаря на основі симптомів

Використання Node.js забезпечує асинхронну обробку запитів, що є важливим для веб-систем з великою кількістю одночасних користувачів.

Таким чином, Next.js дозволяє побудувати повноцінну full-stack систему без розділення на окремі серверні та клієнтські застосунки, що спрощує архітектуру та підтримку проєкту.

Фреймворк Front-end:

Обрано React (у складі Next.js), оскільки бібліотека дозволяє створювати динамічні та масштабовані користувацькі інтерфейси.

React забезпечує:

- компонентний підхід до розробки
- повторне використання UI-елементів
- ефективне оновлення інтерфейсу через Virtual DOM
- зручну реалізацію особистого кабінету та адміністративної панелі

Next.js додатково підтримує Server-Side Rendering (SSR), що покращує продуктивність та SEO-оптимізацію медичного порталу.

Враховуючи необхідність створення складного інтерфейсу з різними ролями користувачів, React є доцільним вибором.

Система керування базами даних (СКБД):

Обрано PostgreSQL, оскільки система потребує чітко структурованих даних та складних зв'язків між сутностями.

Медичний портал включає такі сутності, як:

- Користувачі
- Лікарі
- Пацієнти
- Прийоми
- Симптоми
- Спеціалізації

Між ними існують зв'язки типу 1:1, 1:N та M:N, що ефективно реалізуються в реляційній базі даних.

PostgreSQL забезпечує:

- підтримку транзакцій (ACID)
- цілісність даних через зовнішні ключі
- індексацію для швидкого пошуку
- надійність та масштабованість

Оскільки система працює з персональними та медичними даними, важливою є гарантована цілісність інформації, що забезпечує саме реляційна СКБД.

Інструменти DevOps/Hosting:

Обрано Docker для контейнеризації застосунку, оскільки це забезпечує однакове середовище розгортання на всіх етапах розробки.

Docker дозволяє:

- ізолювати середовище виконання
- уникнути проблем із залежностями
- спростити розгортання

Для автоматизації процесів розгортання використовується GitHub Actions, що забезпечує:

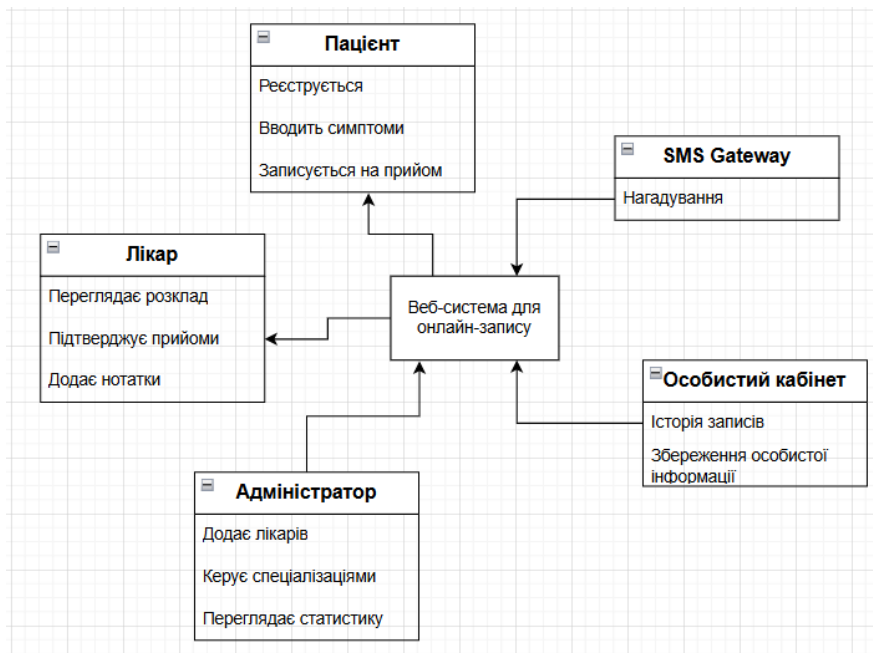
- автоматичне тестування

- CI/CD процес
- контроль версій

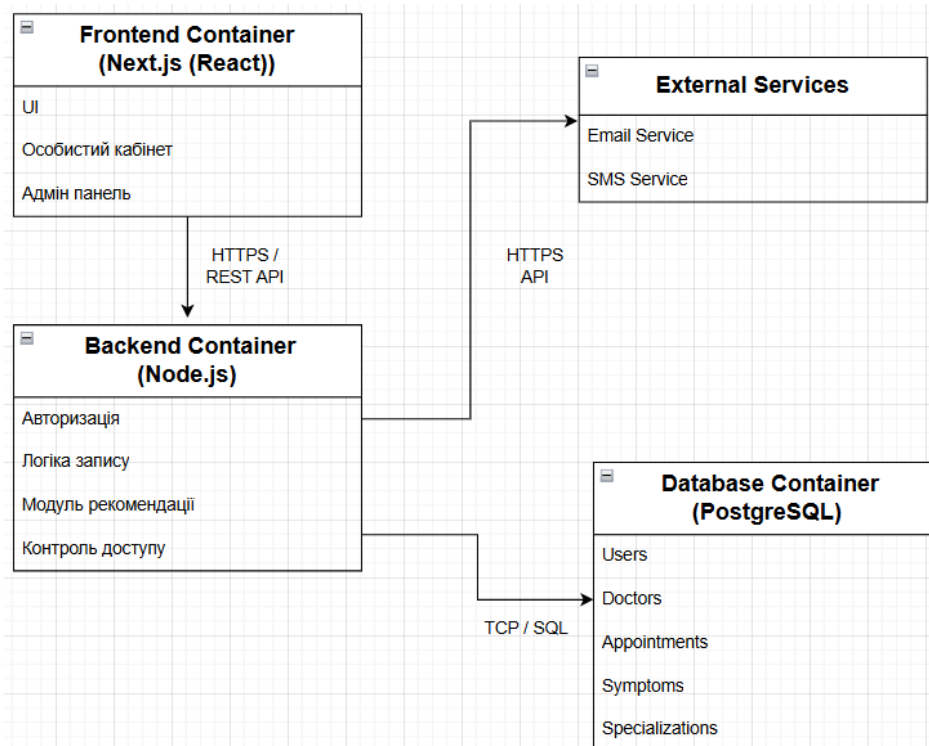
Хостинг може бути реалізований на платформах Vercel або AWS, які забезпечують масштабованість, стабільність та підтримку HTTPS-з'єднання.

2. Архітектурне моделювання за методологією C4 (C4 Model)

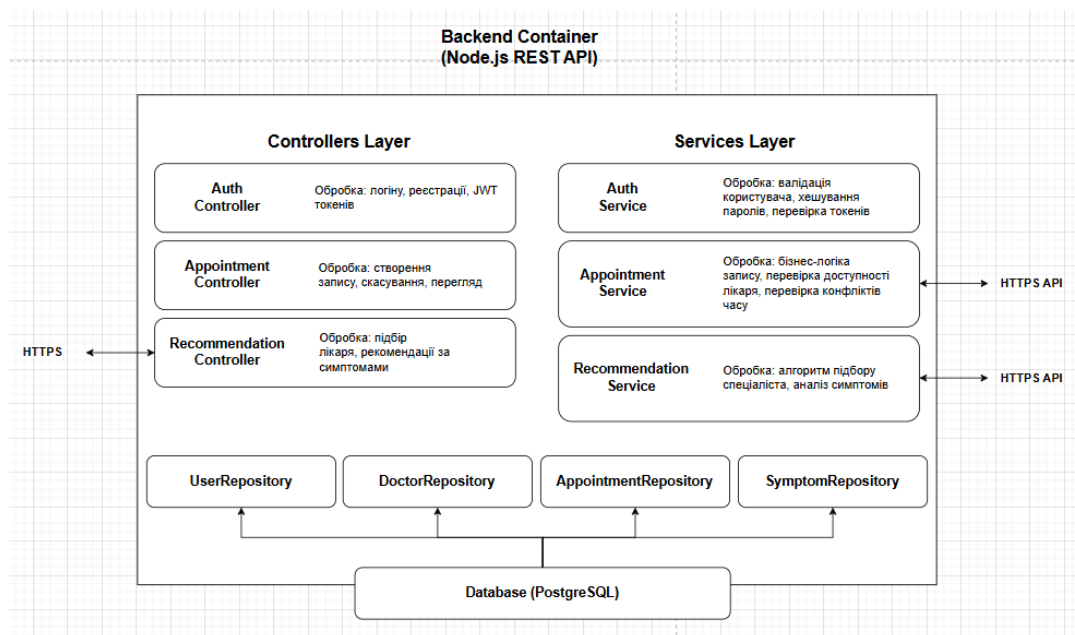
Рівень 1: System Context Diagram (Контекст)



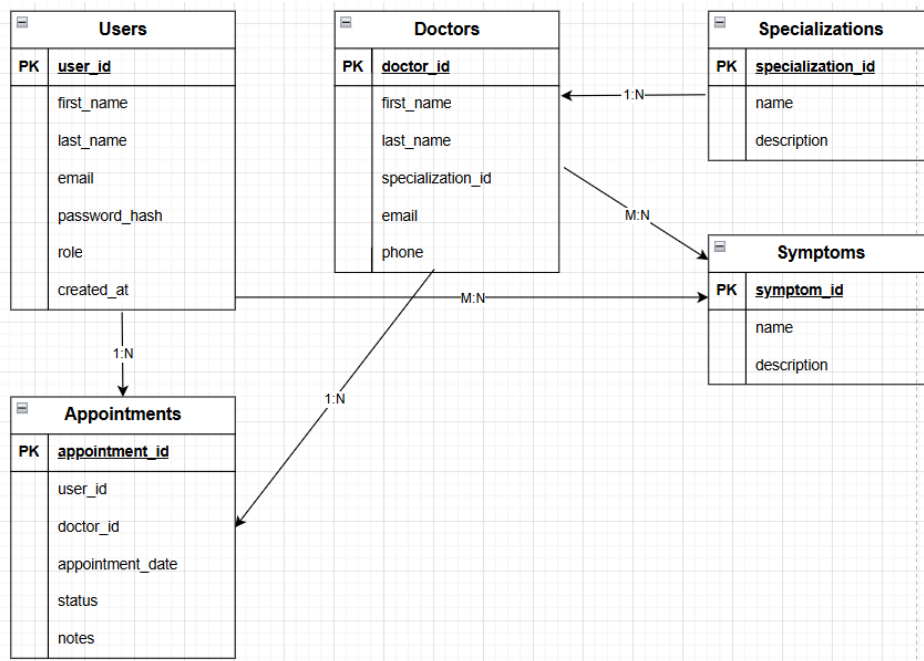
Рівень 2: Container Diagram (Контейнери)



Рівень 3: Component Diagram



3. Проектування бази даних (ER-Model / Data Schema)



Для зберігання даних обрано реляційну СКБД PostgreSQL. Схема бази даних спроектована з використанням ER-моделі. Всі сутності нормалізовано до третьої нормальної форми (3НФ), що забезпечує мінімізацію надлишковості даних та цілісність інформації.

Контрольні запитання

1. Що таке модель С4 і для чого потрібен кожен з її рівнів (Context, Container, Component)?

Архітектура моделі С4 — це ієрархічна структура для візуалізації програмних систем на чотирьох рівнях: контекст, контейнери, компоненти та код. Китайські виробники, зокрема QZY Models, використовують такі структуровані підходи для створення точних, масштабованих архітектурних та промислових фізичних моделей, підтримуючи глобальні ланцюги поставок B2B з можливостями OEM та оптових постачальників.

Архітектура моделі С4 представляє проектування програмної системи на чотирьох рівнях абстракції: Контекст (огляд системного середовища), Контейнери (розбивка застосунків та сховищ даних), Компоненти (внутрішні функціональні модулі) та Код (детальна реалізація). Вона сприяє чіткій комунікації між технічними та нетехнічними зацікавленими сторонами, використовуючи прості, вкладені діаграми, сприяючи спільному та еволюційному архітектурному проектуванню.

2. У чому принципова різниця між реляційними (SQL) та нереляційними (NoSQL) базами даних? Коли варто обирати кожен з них?

Принципова різниця між реляційними (SQL) та нереляційними (NoSQL) базами даних полягає у структурі даних, масштабованості та принципах роботи. SQL базується на суворих таблицях, зв'язках та транзакціях (ACID), підходячи для точних даних, тоді як NoSQL використовує гнучкі моделі (документи, графи) для швидкої обробки великих об'ємів неструктурованої інформації.

- SQL, якщо дані структуровані, а їхня точність є критичною (наприклад, фінанси)
- NoSQL, якщо дані неструктуровані, швидко змінюються або мають великі об'єми, а швидкість розробки та роботи важливіша

3. Що таке теорема CAP (Consistency, Availability, Partition tolerance) і як вона впливає на вибір бази даних?

У той час, як реляційні схеми покладаються на принципи ACID, усі без винятку NoSQL сховища спираються інші принципи, описані в теоремі CAP. Для початку в ній стверджується, що будь-яке сховище даних має три базові властивості:

- **Узгодженість даних (Consistency).** Тобто дані мають бути повними та несуперечливими (у тому числі й у всіх вузлах кластера).
- **Доступність (Availability).** Грубо кажучи, це швидкість відповіді сервера на запит (для запису чи читання).
- **Стійкість до поділу (Partition tolerance).** Це означає, що у разі поділу системи на кілька частин кожна з них, якщо вона доступна, повинна мати можливість працювати автономно, віддаючи коректний відгук та надаючи свої дані. Обрив зв'язків у кластері не повинен впливати на підсумкову роботу.

Теорема CAP повідомляє нам, що із цих трьох компонентів ми можемо отримати лише два. Або в тій же пропорції 2:1 балансувати між цими

складовими: покращення характеристик по одній із властивостей спричиняє погіршення в якомусь іншому. Щоб краще зрозуміти силу цієї теореми, уявіть собі розподілене сховище, в якому ви намагаєтеся без гальм за продуктивністю забезпечити 100% узгоджені дані (читання та запис).

4. Які переваги та недоліки мікросервісної архітектури порівняно з монолітною (на рівні Container diagram)?

Monolithic Architecture — традиційний підхід до побудови програмних продуктів, за якого весь функціонал зосереджений у кодовій базі та виконується в єдиному процесі. Весь стек застосунку, включно з призначеним для користувача інтерфейсом, бізнес-логікою і доступом до даних, інтегрований в одну програму.

Одна з ключових особливостей «моноліту» — наявність єдиного кодового базису. Це означає, що весь вихідний код застосунку знаходиться в одному місці, що полегшує його розуміння і розробку. Запуск і оновлення продукту на монолітній архітектурі вимагає мінімальних зусиль, оскільки всі компоненти знаходяться разом. Це робить процес розгортання відносно простим і зрозумілим для розробників. Завдяки централізованій структурі, процеси моніторингу та налагодження також стають простішими. Розробники можуть використовувати стандартні інструменти для відстеження проблем і поліпшення продуктивності.

Однак у цієї архітектури є й недоліки. У міру того, як монолітний застосунок збільшується в обсязі, можуть виникати обмеження в його гнучкості та масштабованості. Також при внесенні правок може знадобитися зміна всього кодового базису, а це тривалий і трудомісткий процес. У разі збою навіть одного компонента можливі серйозні проблеми у функціонуванні всього застосунку.

Microservice Architecture — підхід до розроблення програмного забезпечення, за якого застосунок розбивається на невеликі автономні сервіси, кожен з яких виконує обмежений обсяг функціонала. Усі складові системи взаємодіють між собою через API (програмний інтерфейс застосунку), що дає змогу їм функціонувати незалежно один від одного.

Основні плюси мікросервісної архітектури для розробки:

- Девелопери можуть працювати над окремими сервісами паралельно — це прискорює процес і дає змогу гнучко масштабувати окремі компоненти продукту.
- У разі збою в одному з мікросервісів інші можуть продовжувати свою роботу, мінімізуючи вплив на весь стек програми — це забезпечує стійкість до відмов.
- Кожен мікросервіс може використовувати свою власну базу даних, що надає більшої гнучкості в управлінні даними — це дає змогу кожному сервісу вибирати найкращий підхід до зберігання даних залежно від своїх вимог.

Серед мінусів Microservice Architecture передусім виділимо децентралізовану структуру, яка може спричиняти складнощі в управлінні комунікаціями та мережею, версіями та оновленнями. Також у разі вибору «мікросервісів» необхідні складніші засоби моніторингу для відстеження стану та продуктивності, якщо порівнювати з «монолітом».